

obstacle_detection

Enten erkennen und umfahren für die Duckiebot Challenge

Nico Sander

21. Juli 2026

1 Ausgangslage und Aufgabenstellung

1.1 Was gegeben ist

Gegeben ist ein Kurs aus Klebeband auf schwarzen Matten. Eine zweispurige Fahrbahn, getrennt durch eine gestrichelte gelbe Mittellinie und außen begrenzt durch durchgezogene weiße Linien, mündet in einen Wendeplatz. Auf dem Wendeplatz stehen Gummienten.



Der Kurs. Unten die Startposition auf der rechten Spur, oben der Wendeplatz mit den Enten. Innerhalb des Wendeplatzes laufen die Fahrbahnmarkierungen nicht weiter.

Der Duckiebot bringt für die Aufgabe eine nach vorn gerichtete Kamera, Radencoder und einen Abstands-sensor mit.

1.2 Zu lösendes Problem

Die Aufgabe lautet: eine Runde durch den Wendeplatz fahren, ohne eine Ente umzufahren.

Gewertet wird nicht die Zeit. Die Geschwindigkeit spielt keine Rolle, und es gibt keinen Vorteil darin, schnell zu sein. Bewertet wird stattdessen, mit wie vielen Enten auf dem Wendeplatz die Umsetzung noch zurechtkommt. Wie die Enten stehen, ist vorher nicht bekannt.

Aus dem letzten Punkt folgt die eigentliche Schwierigkeit: eine feste, vorab geplante Route gibt es nicht, denn sie müsste für eine Anordnung ausgelegt werden, die zur Laufzeit eine andere sein wird. Der Bot muss also während der Fahrt entscheiden, wo er durchpasst.

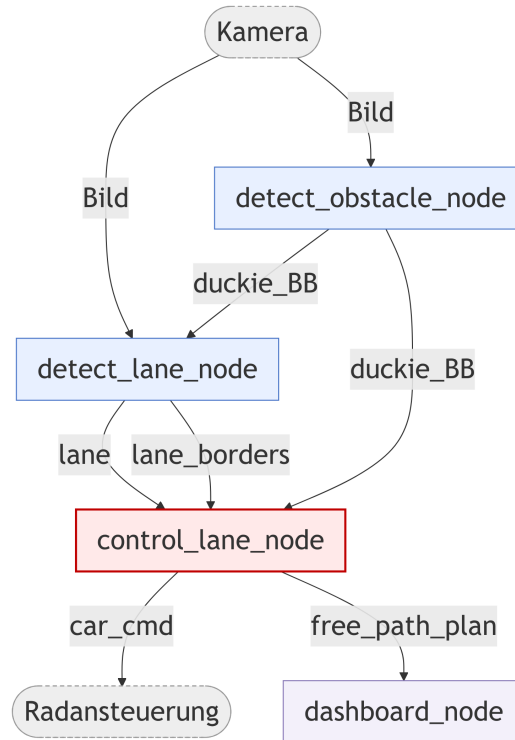
Daraus ergeben sich vier Teilprobleme:

- der Fahrbahn folgen und erkennen, wo sie verläuft,
- die Enten finden und abschätzen, wie weit sie entfernt sind,
- entscheiden, ob ausgewichen werden muss und wohin,
- dabei nicht dauerhaft stehen bleiben, denn eine nicht beendete Runde zählt nicht.

2 Systemüberblick und Kernkomponenten

Das Paket besteht aus vier Nodes. Drei davon bilden zwei Schichten: die *Wahrnehmung* wertet das Kamerabild aus und liefert daraus die Lage der Fahrbahn und die Positionen der Enten, die *Fahrentscheidung* verrechnet beides zu einem Fahrbefehl. Der vierte Node ist ein Dashboard, das rein der Fehlersuche dient und am Fahrverhalten nichts ändert.

Von den vorhandenen Sensoren wird nur die Kamera genutzt; Encoder und Abstandssensor bleiben ungenutzt. Der Regler entscheidet jeweils aus dem aktuellen Kamerabild neu und führt keine Karte der Umgebung mit.



Der ROS Node Graph, auf den Hauptdatenfluss reduziert. Kästen sind Nodes, Pfeile sind Topics, gestrichelt sind Kamera und Radansteuerung als externe Beteiligte. Blau ist die Wahrnehmung, rot die Fahrentscheidung, violett die Anzeige. Die Debugbilder ans Dashboard sind der Übersicht halber weggelassen.

2.1 Wahrnehmung

detect_lane_node.py Segmentiert das Kamerabild in weiße und gelbe Fahrbahnmarkierungen und leitet daraus den Querabstandsfehler zur Spurmitte und die Spaltenpositionen der beiden Linien ab. Das U-Net dahinter ist dasselbe wie in den Challenges 1 und 2 und dort beschrieben; neu sind die folgenden drei Punkte.

Für diese Challenge kam eine Schutzmaßnahme dazu: bevor die Linienpositionen berechnet werden, werden die aktuellen Entenboxen aus der Segmentierungsmaske entfernt. Eine gelbe Ente auf der Fahrbahn sieht dem gelben Mittelstreifen sonst ähnlich genug, um die Spurlage zu verfälschen, und zwar genau in dem Moment, in dem der Bot auf sie zufährt.

Zwei weitere Signale entstehen hier und werden zusammen mit den Linienpositionen verschickt. Verteilen sich die Pixel einer Linie über einen großen Teil der Bildbreite, statt einer Spalte zu folgen, so liegt sie quer im Bild und wird als frontal angefahren gemeldet: sie ist dann keine Spurbegrenzung, der man folgen kann, sondern eine Wand. Und erscheinen die beiden Linienfarben in vertauschter Reihenfolge, ist der Bot verkehrt herum in der Spur unterwegs.

detect_obstacle_node.py Erkennt Enten mit einem selbst trainierten YOLOv11 Modell und veröffentlicht alle Bounding Boxen als JSON. Der Node trifft keine Fahrentscheidung, er liefert nur strukturierte Hindernisinformation.

Wie **detect_lane_node** schützt er sich mit einer Wiedereintrittssperre: liegt die Inferenz hinter dem Kamerabildstrom zurück, wird das neue Bild verworfen statt eingereicht. Ein verworfenes Bild ist besser als ein veraltetes, denn gehandelt wird auf der aktuellen Pose.

2.2 Fahrentscheidung

control_lane_node.py Der zentrale Node und das Herz des Pakets. Er ist als Regler nach dem Prinzip „Folge der Lücke“ mit einem kleinen Zustandsautomaten aufgebaut. Das Spurfolgen liefert dabei die Route einschließlich der Wende; die Ausweichschicht verändert nur die Lenkung und klemmt sie hart in den Korridor zwischen den sichtbaren Linien.

Aufgebaut ist er als Zustandsautomat nach dem EVA Prinzip: Sensordaten hinein, Zustand bestimmen, genau ein Fahrbefehl hinaus. Weil der Fahrbefehl an einer einzigen Stelle entsteht, lässt sich dort garantieren, dass er immer aktiv ist. Gibt es keinen Weg nach vorn, dreht der Bot sich also auf der Stelle und sucht eine Lücke, statt anzuhalten. Findet er längere Zeit keine, lockert er schrittweise seine Anforderungen, bis eine Lücke als befahrbar gilt.

Die eigentliche Fahrlogik steckt in einer eigenen Klasse. Der Node sammelt die eingehenden Nachrichten ein und ruft sie zehnmal pro Sekunde auf; sie liefert daraus Geschwindigkeit und Drehrate.

Startsperre Der Regler fährt erst los, wenn alle drei Eingangsdaten mindestens einmal eingetroffen sind, denn ein noch stummes Topic ist von „keine Ente voraus“ nicht zu unterscheiden.

2.3 Bedienung und Visualisierung

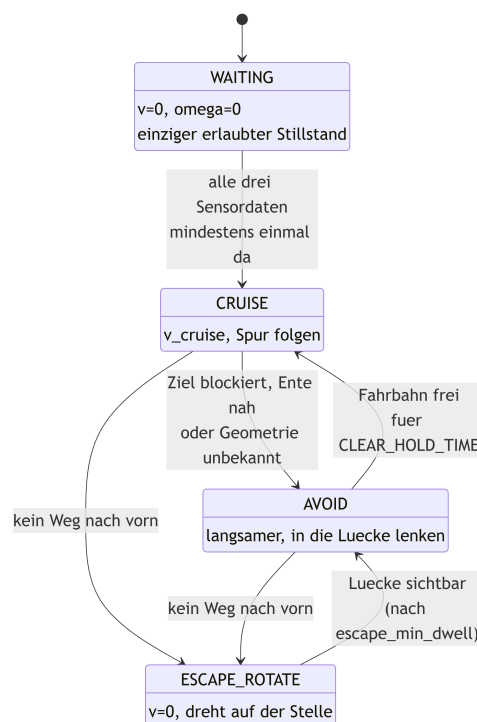
dashboard_node.py Fasst die Debugbilder in einer Ansicht zusammen: Kamerabild mit erkannten Linien, die weiße und die gelbe Maske sowie ein Overlay der Planung. Im Overlay sind die durch Enten gesperrten Bereiche rot, die freien grün, die gewählte Lücke cyan umrandet und die aktuelle Zielspalte blau. Damit ist auf einen Blick zu sehen, warum der Regler gerade so entscheidet, wie er entscheidet.

Trockenlauf Die Startdatei kennt ein Argument, das den gesamten Stapel startet und planen lässt, aber den Fahrbefehl nicht veröffentlicht. Der Bot steht dabei still, während das Dashboard vollständig gefüllt bleibt. Das ist die Betriebsart, in der die Parameter abgestimmt werden.

3 Zustands- und Ablaufdiagramme

3.1 Fahrzustände

Der Zustandsautomat ist bewusst klein gehalten. Alle Übergänge liegen in einer einzigen Funktion, getrieben von skalaren Signalen und Mindestverweilzeiten, sodass es keine zweite Stelle gibt, die dieselbe Entscheidung anders treffen könnte.



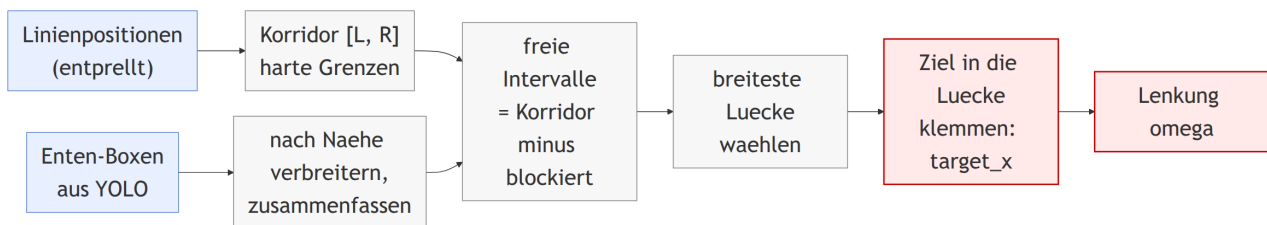
Die Fahrzustände des `control_lane_node`. `WAITING` ist die Startsperrung und der einzige Zustand, in dem der Bot steht. `ESCAPE_ROTATE` fährt nicht, dreht aber immer, womit die Invariante gewahrt bleibt. In diesen Zustand führen außer einer versperrten Fahrbahn auch die beiden Sonderfälle aus Abschnitt 2.1: eine quer angefahrene Markierung und eine verkehrt herum befahrene Spur. Beide werden wie ein Hindernis behandelt, weil Drehen in beiden Fällen die richtige Antwort ist und die Lage zugleich auflöst.

3.2 Lückenplanung pro Takt

In jedem Takt wird aus den beiden Sensorbildern eine einzige Zielspalte berechnet. Die sichtbaren Linien sind dabei harte Grenzen: der Zielpunkt wird immer in den Korridor geklemmt, sodass keine Markierung überfahren wird.

Hat der Bot eine Lücke gewählt, bleibt er für eine Mindestzeit dabei, auch wenn sie zwischenzeitlich zu schmal misst: die Boxen wachsen beim Heranfahren, sodass jede Lücke zwangsläufig enger wird und das Manöver sonst auf halbem Weg abbrechen würde. Nur eine wirklich nahe Ente direkt voraus hebt diese Bindung sofort auf.

Bevor die freien Intervalle bestimmt werden, wird jede Entenbox verbreitert, und zwar umso stärker, je näher die Ente steht. Eine feste Verbreiterung im Bild wäre nur in genau einer Entfernung richtig: die scheinbare Breite eines Objekts fällt mit dem Kehrwert der Entfernung, und der Bot selbst erscheint aus wenigen Zentimetern um ein Vielfaches breiter im Bild als aus einem halben Meter. Sicherheitsabstände sind deshalb in Zentimetern angegeben; eine auf dem Kurs gemessene Kennlinie übersetzt die Unterkante einer Box in eine Entfernung und diese erst im Takt in Bildbreite.



Die Lückenplanung. Aus den freien Intervallen wird das breiteste gewählt und das Ziel hineingeklemmt. Eine Hysterese dämpft das Umschalten zwischen zwei ähnlich breiten Lücken, damit der Bot nicht zwischen links und rechts pendelt.

4 Konfiguration

Die abgestimmten Größen liegen in `config/` als JSON, getrennt nach Regler und Detektor. Der Regler kann seine Parameter zusätzlich zur Laufzeit entgegennehmen, was das Abstimmen auf dem Bot deutlich verkürzt.

Zum Abstimmen gehört außerdem ein Werkzeug, das eine aufgezeichnete Fahrt mit geänderten Parametern erneut durchrechnet. Auf dem Roboter liefert jede Sitzung nur eine verrauschte Stichprobe; dieselbe Aufzeichnung erneut zu rechnen, macht daraus einen kontrollierten Vergleich.

Am stärksten auf das Verhalten wirken diese fünf Größen:

- `safety_margin_cm` (2 cm): Abstand, den der Bot zusätzlich zu seiner halben Breite um jede Ente herum freihält. Der größte Hebel dafür, wie eng er an einer Ente vorbeifährt.
- `gap_min_cm` (2,4 cm): wie viel breiter als dieser Mindestabstand eine Lücke sein muss, damit sie als befahrbar zählt.
- `front_block_ymax` (0,8): ab welcher Nähe eine Ente direkt voraus als Sperre gilt und der Bot dreht, statt weiterzufahren.
- `avoid_min_dwell` (1,6 s): wie lange der Bot an einer einmal gewählten Lücke festhält.
- `escape_relax_after` (1,5 s): nach welcher erfolglosen Drehzeit er beginnt, Mindestbreite und Sicherheitsabstand zu lockern.